How our decoders evolved

Collaborator brief · October 30, 2026

In one page

We solve PDEs with a handful of unknowns instead of one per mesh point. A **decoder** turns those few unknowns into the full field, and the reduced solver adjusts them until the decoded field satisfies the equations. The question throughout has been: **which decoder gives an accurate solve that is also cheap?**

- **Grid tensor decoders (CP).** Cheap, but tied to one grid, and they did not improve under refinement.
- **Coordinate networks (SIREN, attention, FiLM).** Can be queried anywhere and reconstruct well, but are expensive inside every solver iteration and sometimes solve badly.
- **Separable learned bank + small head.** Evaluate the spatial functions once and cache them, so a small network only picks coefficients. This became the main architecture.
- **Is the head needed?** Correction directions and solving directly for bank coefficients (ordered prefixes) became the main path. Tuned POD stayed a strong competitor.
- **Solver and comparisons.** Off-mesh quadrature and better time stepping improved accuracy, but a tuned conventional solver often reaches the same accuracy faster.

Bottom line.

- **Supported:** reduced solves whose cost does not grow with the output mesh, and evaluation at arbitrary points, in the qualified Burgers settings.
- **Not supported:** a general speed or accuracy advantage over tuned spectral solvers or tuned POD hyper-reduction.
- **Scoped wins:** moving-frame models for translating Navier–Stokes flows.
- **Main lesson:** better reconstruction is not the same as a better PDE solve.

Want more detail? Per-architecture write-ups, numeric tables, retractions and a glossary are in the full history report. Every code file is mapped to its original branch and commit in the code index.

The four decoder forms

$$\text{CP (grid):} \quad u(i,j;z) = b + \sum_a h_a(z) X_{a,i} Y_{a,j}$$

$$\text{Coordinate network:} \quad u(x;z) = f_\theta(x,z)$$

$$\text{Learned bank + head:} \quad u(x;z) = \sum_a g_a(x) h_a(z)$$

$$\text{Ordered bank (current):} \quad u(x;c) = B_r(x)\, c$$

The last form is linear in its online unknowns even though a neural network learned B_r. A nonlinear head inside a fixed bank can never leave that bank's span.

Timeline

Period	What we tried	What we learned
May	CP, LinearCP, ViT, cross-attention, SIREN, affine maps	Good reconstruction does not guarantee a good solve
July	Frozen POD, POD + residual MLP, faster CP heat	Linear bases are serious competitors
August	Concatenated / FiLM coordinate nets, latent stepping, grouped FiLM	Mesh transfer works; cost and solve geometry limit it
Aug–Sep	Cached bank + head, head variants, corrections, ordered prefixes	Separate bank error, head error and solve error
Sep–Oct	Moving-frame and divergence-free banks for Navier–Stokes	Useful for translating flows only
October	Wider / smoother banks, off-mesh quadrature, better stepping, stronger baselines	Mesh-independence holds; general speed win does not
Now	Obstacle-flow ROM, proposed local tree bank	Both still unproven

The milestones that mattered

CP and LinearCP — the starting point. An MLP picks weights for fixed grid-factor products. Cheap to evaluate, but the grid-tied factors were the resolution bottleneck. Adding a linear skip did not fix that. cp code · linearcp code

Modulated SIREN — the key failure. Accurate autoencoders still gave bad cold-start solves because of a spurious attraction basin, not rank loss. Making the output affine in the latent was the first glimpse of the cached-bank idea. siren code · history

FiLM with latent stepping — first true ROM with coordinate nets. Solving for the latent from the PDE exposed problems that supervised reconstruction hid. Smooth weak tests fixed pointwise-residual blow-up. Grouped FiLM reached about 0.5% (Poisson) and 1% (Burgers) mean solve error. film code · latent stepping · grouped FiLM

Cached learned bank + head — the core architecture. Move the coordinate network out of the inner solve and keep only a small head online. This also separated two bottlenecks: the bank's capacity and the head's. code · design

Ordered bank prefixes — the current Burgers path. Order the trained bank once and solve directly for the first r coefficients. This gives a deployment-time accuracy/cost knob. Corrections on top of a head were the intermediate step. ordered · stepping · corrections

Moving frame — the clearest scoped win. Solve for a shift and a shape, $u(x) = \tilde{u}(x - c; z)$. Centering made translating structures compact, and here a nonlinear head did help. shift · shift head · curl bank

October solver work — biggest accuracy gains, honest comparisons. Off-mesh quadrature and better time integration helped more than further decoder changes. Tuned spectral and POD baselines removed the broad speed claims. Burgers report · refinement report

Everything else we tried

Variant	Outcome	Code
Mirrored ViT decoder	Dropped: reconstructs, but solves fail	vit
Cross-attention INR	Dropped: needs snapshot tokens unavailable at solve time	xattn
Default-field / anchored SIREN	Mixed: cold-start fix lost when scaled up	default_siren
Multiresolution / FAS (May)	Not established: used truth to pick the best cycle	lab log
Frozen POD	Kept as baseline: often the strongest competitor	pod_hr
POD + residual MLP	Historical: mixed gains	podmlp
Multistage residual fitting	Dropped: single-function precision did not transfer	staged
Residual-FiLM	Dropped: missed accuracy targets	film_arch
Wider / deeper heads, latent Fourier features	No gain on solved error	headfit
Protected-anchor head	Dropped: geometry fine, accuracy gates failed	anchor
Quadratic head	Partial: better mean (4.7% vs 5.5%), worse worst case	quadratic
Shared encoder for head training	Dropped: inconsistent across repeats	encoder
Mixture of local heads	Dropped: no validation gain	mixture
Modified CP (latent-dependent factors)	Dropped: no matched win; wave physics wrong	modcp
Variable-projection bank training	Kept: good heat banks	heat_vp
Stokes and wave banks	PDE-specific; old wave claims superseded	stokes
Masked sparse autoencoder	External baseline, reproduced	masked
Classical quadratic manifold	Baseline: beats linear at equal size, at extra cost	qman
Fixed-frame Navier–Stokes bank + head	Dropped: no advantage over POD	ns2d
Wider / Sobolev-smooth banks	Better fits, not better solves	wide
Neural-operator controls (FNO, DeepONet, …)	Comparators, not our decoders	operators
Same-sample reconstruction vs interpolation	Dropped: missed both accuracy gates	report
Prefix coarse-to-fine initialization	Dropped: failed cost check	prefix
Separate-head latent continuation	Dropped: 2.6–3.4× slower than direct	transfer
Shared-decoder latent hierarchy (Oct 29)	Dropped: failed cost check	lab log

Where we stand and what's next

- **Claim carefully.** Mesh-independent reduced cost, point queries and conditional accuracy gains are supported. A general neural speed advantage on Burgers is not.
- **Obstacle flow (update, Oct 29).** References for the fixed channel are admitted and a neural bank was trained. Its representation and solve accuracy missed the targets. Classical-basis comparisons are unfinished.
- **Next architecture: global + local tree bank.** Add local correction banks only where they are needed. This is untested. Check first with simple local POD or polynomial bases; train neural features only if localization helps. plan
- **Multilevel initializers.** Three bounded attempts failed on cost. Try another only with a concrete reason it would cut fine-level work.